

Indian Institute of Technology
Department of Computer Science

CS202: Software Tools and Technologies Assessment 2

Jaskirat Singh Maskeen and Aarsh Wankar
(23110146, 23110003)

Instructor: Prof. Shouvick Mondal

September 8, 2025

Contents

1	Evaluation of VAT using CWE-based Comparison	1
1.1	Introduction	1
1.2	Setup and Tools	1
1.3	Methodology and Execution	1
1.4	Results and Analysis	6
1.5	Discussion and Conclusion	10

Lab 1

Evaluation of Vulnerability Analysis Tools using CWE-based Comparison

[Github Repository for Lab 6](#)

1.1 Introduction

Git is a version control system that helps us manage the versions of our projects either locally or on some Git provider such as GitLab or GitHub. The primary advantage of using GitHub or any such provider is that we can collaborate with other users and share our code online with other potential users. This sort of framework supports active development and constant feedback, as anyone can create an Issue or fork our repository and provide pull requests to improve or fix issues with our code.

The main goal of this lab is to understand version control systems (VCS), set up and configure git, verify the installation, and run simple Git commands to push files.

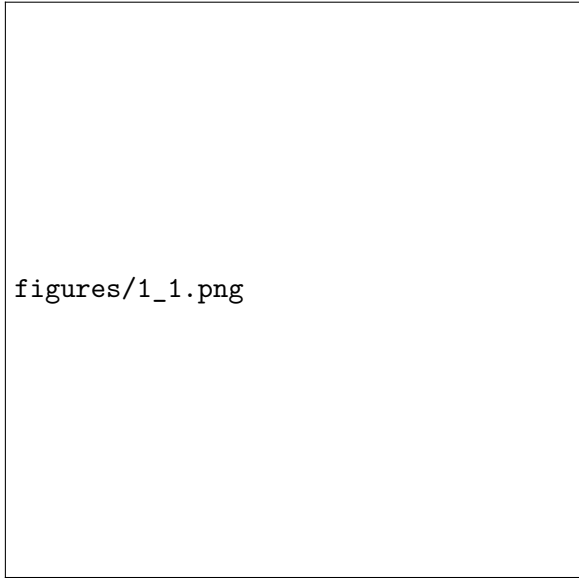
1.2 Setup and Tools

I have Git, Python, and VSCode installed. Since I have already used GitHub before, I have it preconfigured and installed, so I will skip the screenshots for installation details. I will use Git CLI on the command prompt and GitHub Web Interface to create repositories and run workflows. I will also use pylint to perform static type checking. `git --version` gives `git version 2.45.2.windows.1` as the output. While logging into git using the CLI, we first have to sign in using username and password, and verify on GitHub Mobile, as I have 2FA enabled. I also confirm if the current credentials are set (Fig. 1.1). The `--global` flag is used because I have two GitHub accounts on my laptop, and I want to verify the global configuration. These commands were looked through using <https://git-scm.com/docs/git-config>. My working directory is named STT_Lab_1.

1.3 Methodology and Execution

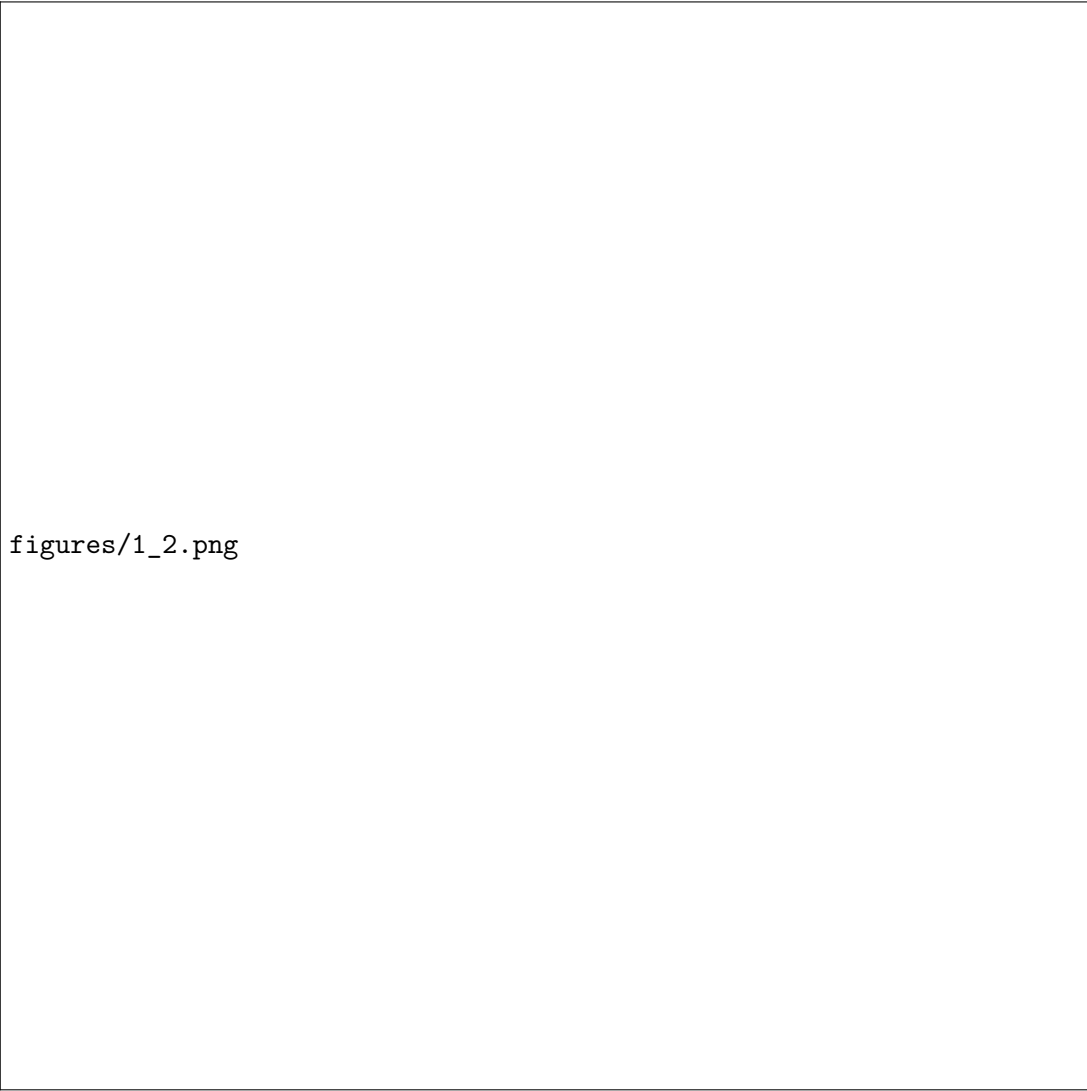
I first initialized a repository in my working directory using `git init`. I have put in a sample file named **trib.py**, which calculates the n^{th} Tribonacci number. I also added a **readme.md** file, which will get rendered on the link of this repository. I also plan to have a single repository on GitHub, with a new branch for each lab to avoid clutter. Hence, I will also rename the branch to **lab1**,

I can check the status of the repository using `git status` (Fig. 1.2), and add files one by one using `git add filename`, or all unstaged files (i.e., which have a delta), at once using `git add .`, I can commit these changes using the command `git commit -m "Added files"`, where "Added files" is a commit message (Fig. 1.3). `git log` is also a useful command to check the history of the commits being made. Now I create a GitHub repository, **CS202-STTCSE**, which will be the repository where I will upload all the labs, just on their respective branches. GitHub Link: <https://github.com/jsmaskeen/CS202-STTCSE/tree/lab1>.



```
figures/1_1.png
```

Figure 1.1: Configuring the git CLI



```
figures/1_2.png
```

Figure 1.2: Initializing a git repository

I then added the local repository to GitHub using the commands,
`git remote add origin git@github.com:jsmaskeen/CS202-STTCSE.git,`

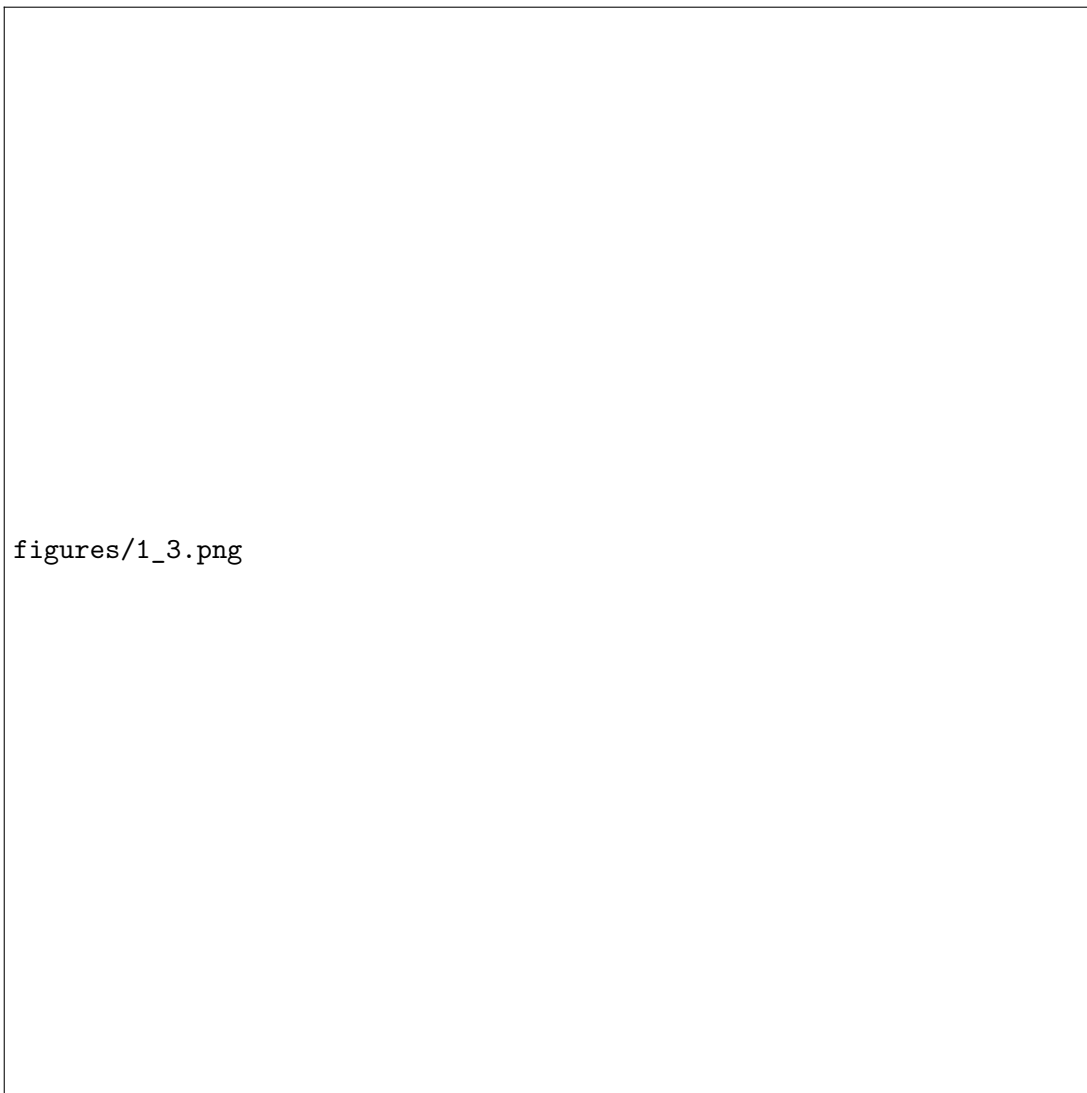


Figure 1.3: Committing to **lab1** branch.

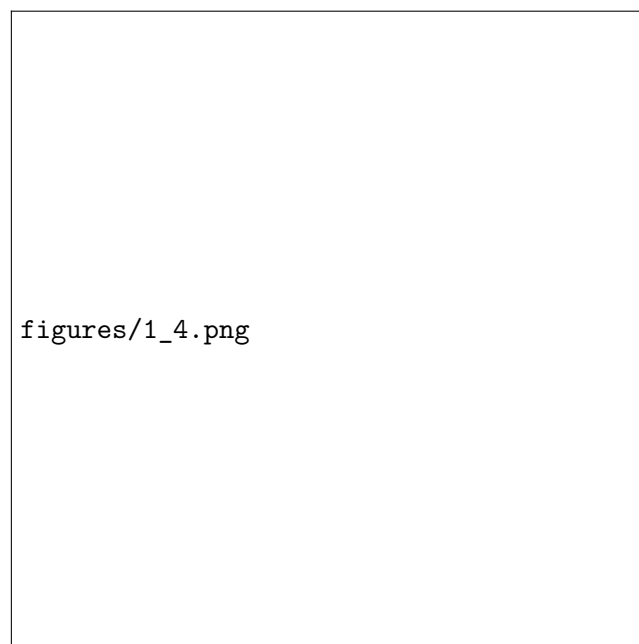


Figure 1.4: `git log` command

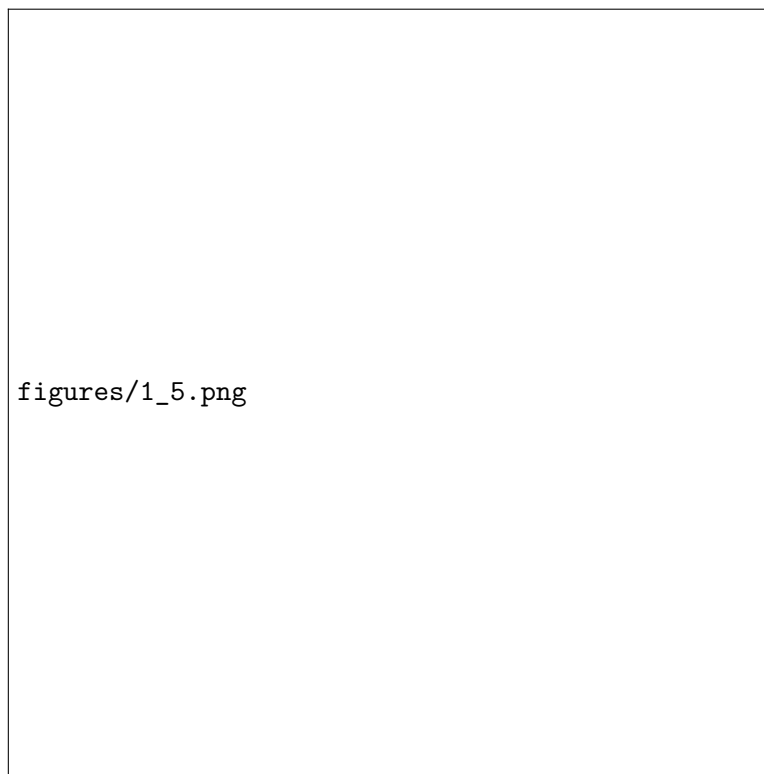


Figure 1.5: GitHub repository creation

Followed by `git push -u origin lab1`.

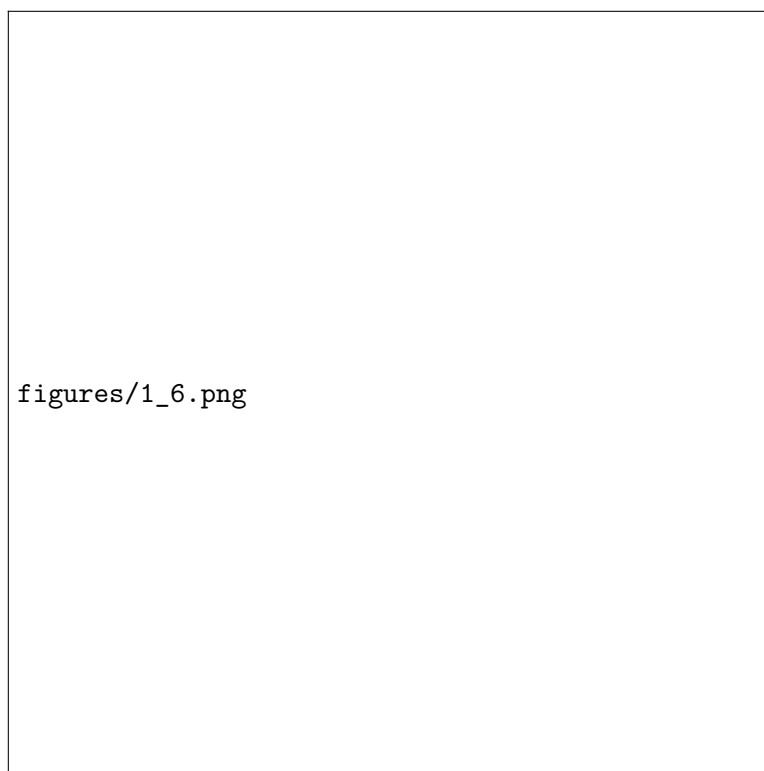


Figure 1.6: Pushing the code to remote repository on GitHub

So, in this way, my local repository is now on GitHub. Now, if I do any commits and push them locally, they will reflect on GitHub as well. For example, I add another file, say **mergesort.py**. I will stage and commit the file, but not push it. Then I will clone this repository elsewhere, and to demonstrate the pull command, I will push the **mergesort.py** file after I have cloned the repository. Please see the sequence of

commands carefully ((Figures 1.7, 1.8, 1.9, 1.10, and 1.11)).

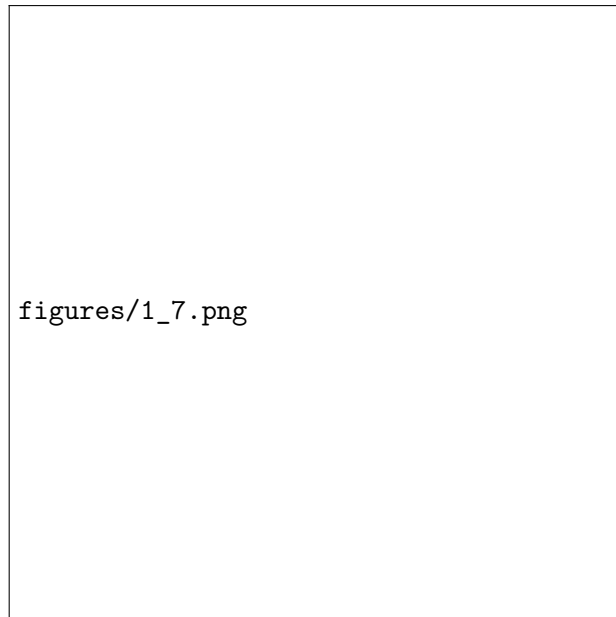


Figure 1.7: **Step 1:** Stage the **mergesort.py** and commit

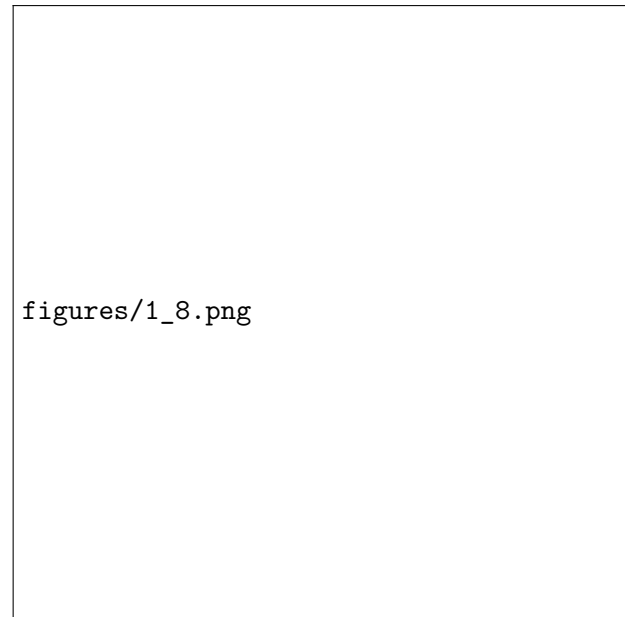


Figure 1.8: **Step 2:** Clone the repository elsewhere

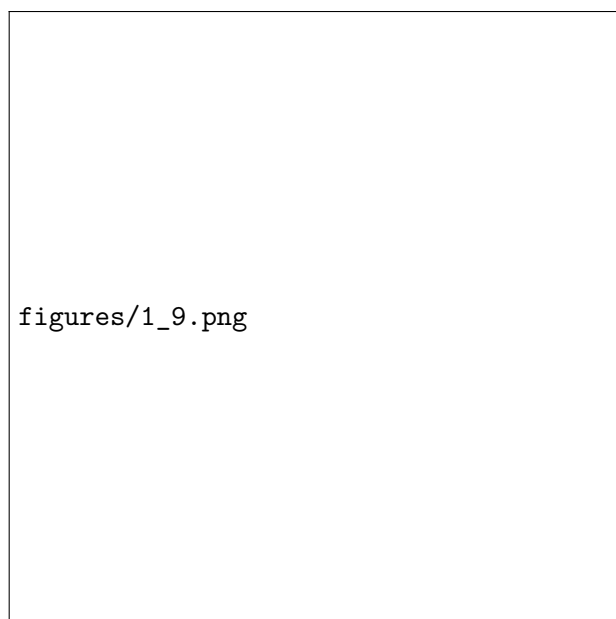


Figure 1.9: **Step 3:** Now I push to GitHub.

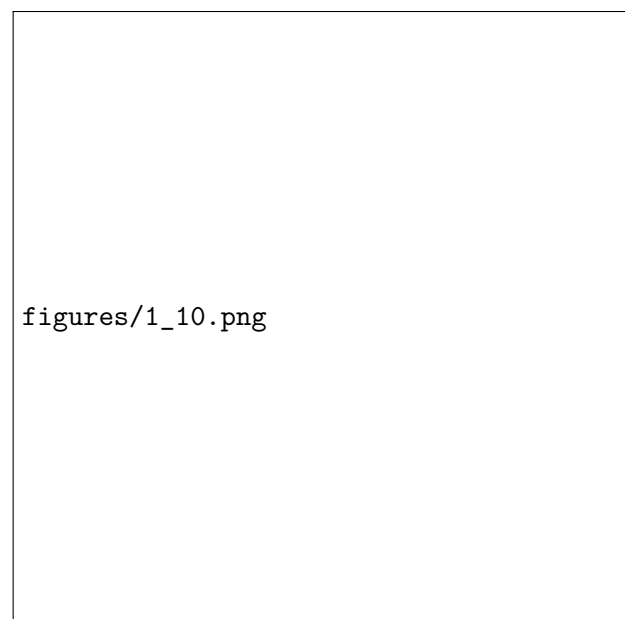


Figure 1.10: **Step 4:** Check with the cloned repository (git fetch, git status)

This demonstrated the `git pull` as well as `git clone` commands. Now I have to create a pylint workflow, so I have deliberately kept some issues in the **mergesort.py** file, such as unused imports, unused functions, and an unused while loop. I hope to see pylint detect all these issues.

To create a pylint workflow, I configured the GitHub action corresponding to pylint.

Once I clicked on configure, a **pylint.yml** file appeared in editing format under a new folder named workflows (Fig. 1.12). Once I committed and pushed this file on the GitHub UI (This means I will have to do `git pull` on local (Fig. 1.13), to stay up to date), the action ran and gave a few errors.

Whenever the action fails, we are greeted with a **✗** (a red cross) (Fig. 1.14), and whenever it passes, we are greeted with a **✓** (a blue tick).



Figure 1.11: **Step 5:** Check the file structure before and after pulling. (git pull)

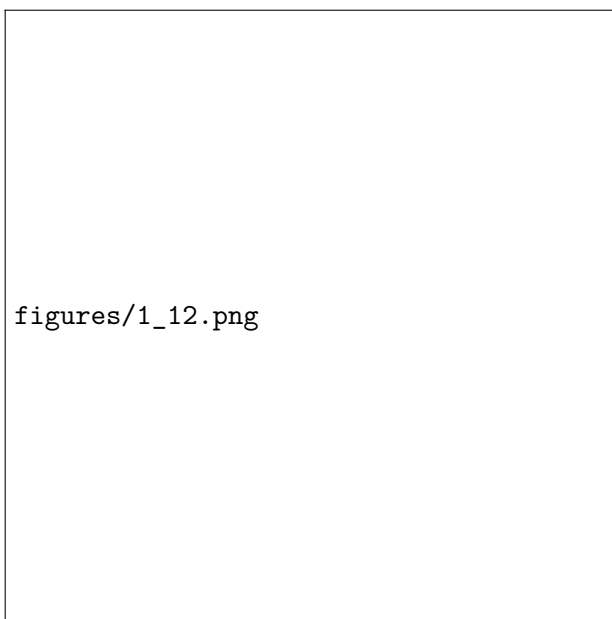


Figure 1.12: Pylint workflow

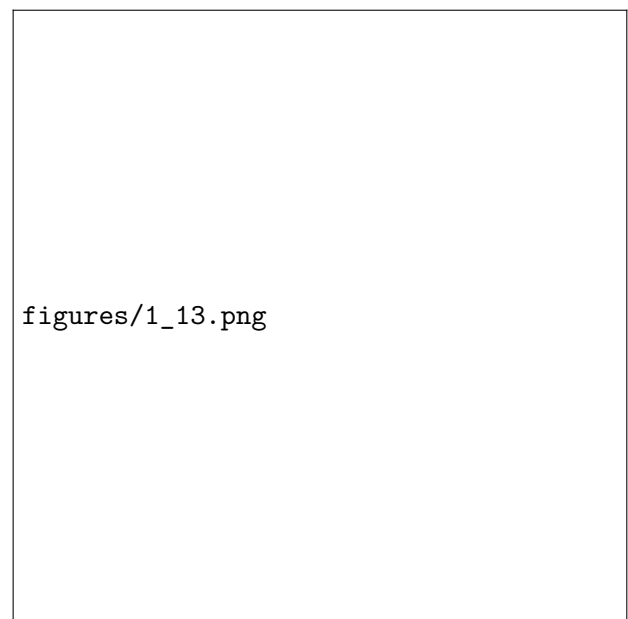
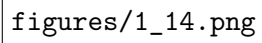


Figure 1.13: Pulling on local to stay up to date

1.4 Results and Analysis

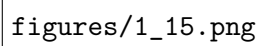
Since I uploaded an erroneous file (which executes correctly, but has unused code, and should fail static checking) (Fig. 1.15).

Looking carefully at the details, we see multiple issues (Fig. 1.16, Fig. 1.17). However, the workflow was not able to detect a few issues, for example:

A screenshot of a workflow interface showing a progress bar and status indicators, indicating that the workflow is currently running.

figures/1_14.png

Figure 1.14: Workflow is running

A screenshot of a code editor showing the contents of a file named mergesort.py. The code contains several syntax errors, which are highlighted in red. The errors include missing closing brackets and incorrect indentation.

figures/1_15.png

Figure 1.15: **mergesort.py** file (with errors)

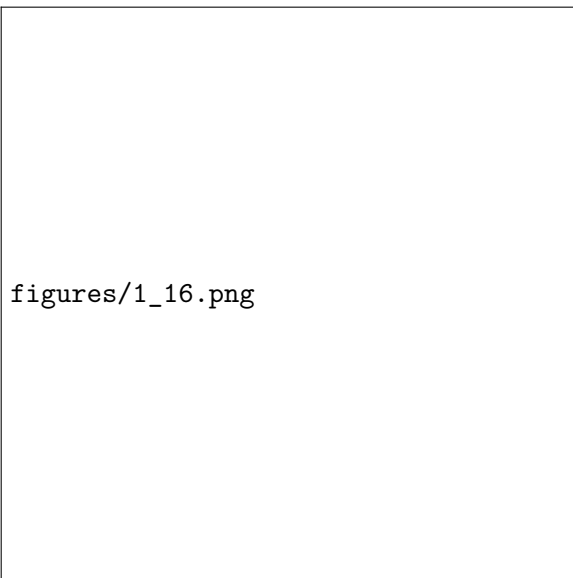


Figure 1.16: Failed Checks

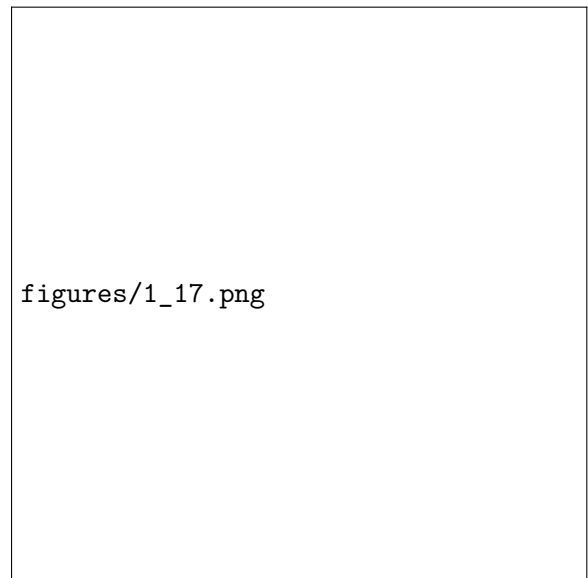


Figure 1.17: Issues found by workflow.

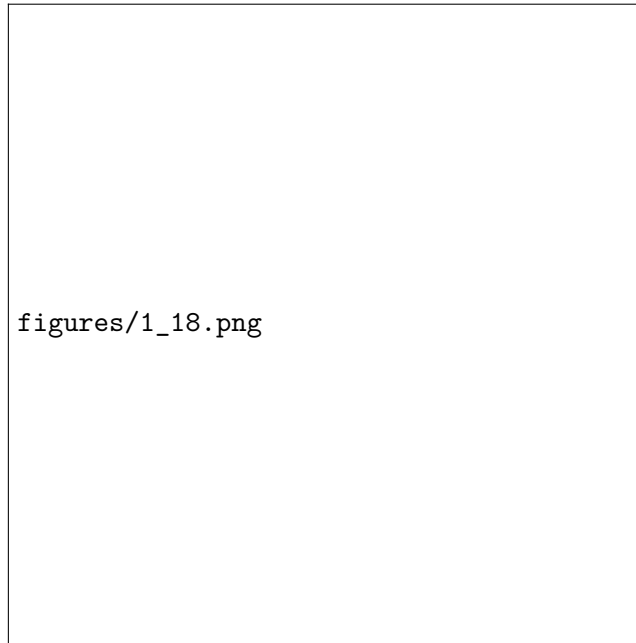


Figure 1.18: Corrected Code

In Fig. 1.18 we have the same `if` loop repeated (which swaps left and right), which serves no function. So, it should be kept only once. [But static analysis cannot capture that]. It was also not able to detect the unused function. To fix the issues, I have edited the docstrings, multiple statements in a single line, and line too long issues.

After pushing with these changes, we see that our tests have passed (Fig. 1.19).

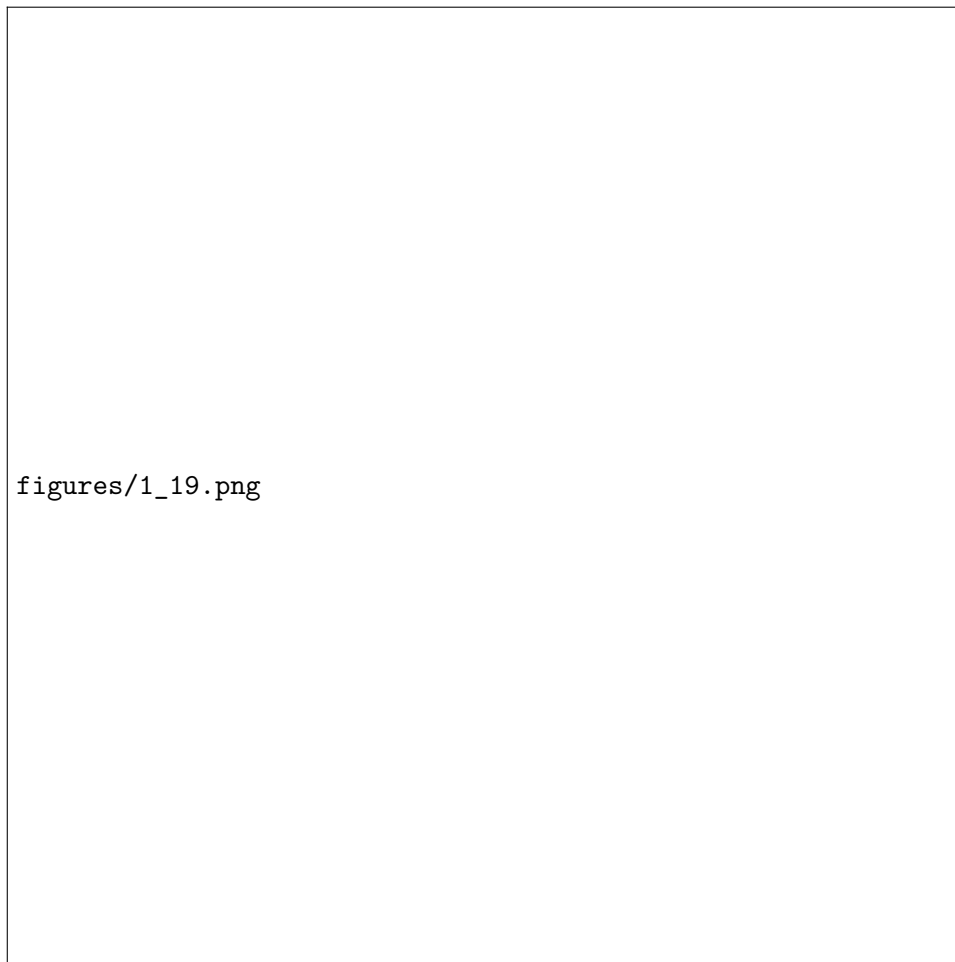


Figure 1.19: All checks passed after fixing the **mergesort.py** file.

1.5 Discussion and Conclusion

This was a really interesting and useful lab session. Using git/GitHub will help us manage our code and collaborate with others; specifically, GitHub actions can configure our workflows, be it static checking or deploying a website after some post-processing. Maybe we write a library that requires a compilation, and various languages comprise the code base, so we can have a GitHub action to compile in the order we like and merge the files, to finally create an executable, for different OS. I did not face any difficulty in this lab; however, while configuring git, I had two accounts. I configured it with SSH tokens, so to set origin for a repository in my main account, I do `git remote add origin git@github.com:jmaskeen/repo.git`, and for my other account, I do `git remote add origin git@github.com:unofficial:jsxxx/repo.git`. Another thing I learnt, if we have ssh enabled, we can check if we are authenticated to GitHub via the command `ssh -T git@github.com`. Lastly, pylint was good, but it can only check issues statically. One mode of improvement can be to have two or more workflows, which also perform static checking, and they do not yield conflicting results, to find any issues missed by pylint (for example, the unused functions).